

RELATÓRIO EXECUTIVO — 1ª PARCIAL (O PLANEJAMENTO)

Projeto TrueCompass · Universidade São Judas Tadeu · Matemática Computacional Aplicada

SEÇÃO 1 | CAMPO DE IDENTIFICAÇÃO

Nome do Projeto	TrueCompass — sistema de mobilidade urbana
Problema Computacional a Resolver	Determinar, sob restrição de latência, qual motorista disponível alcança o passageiro no menor tempo real de deslocamento e qual a rota de menor custo até o destino, sobre uma malha viária de grande porte com ruas de mão única.
Conceitos Matemáticos Aplicados	Teoria de Grafos (Dijkstra, A*, grafo reverso e dualidade many-to-one); Árvores (Quadtree); Ordenação Topológica (algoritmo de Kahn sobre DAG); Indução Matemática (corretude); Análise de Recorrência (Teorema Mestre).
Integrantes do Grupo (Nome e RA)	Bianca Murcela — RA 8261104797 · Carolina Chaves — RA 826182449 · Carolina Yukari — RA 8261110898 · Gabryela Araujo — RA 826199094 · Kauan Sousa — RA 8261113181 · Wildiner Lopes — RA 8261102715
Data da Submissão	21/09/2026

SEÇÃO 2 | RESUMO EXECUTIVO

O TrueCompass é uma plataforma de mobilidade urbana que conecta passageiros a motoristas, sustentada por uma camada algorítmica de implementação própria em Python. O sistema resolve dois problemas de decisão sob restrição de tempo: identificar qual motorista disponível alcança o passageiro no menor tempo real de deslocamento e determinar a rota de menor custo até o destino, ambos sobre a malha viária modelada como grafo ponderado direcionado. São beneficiados passageiros, motoristas parceiros e operadores do setor de transporte individual urbano. O diferencial matemático repousa em três decisões: o emprego do algoritmo A* com heurística haversine admissível, que preserva a otimalidade do Dijkstra reduzindo 3,2 vezes os nós expandidos; a exploração da dualidade $\text{dist}_G(p, m) = \text{dist}_G(m, p)$, que resolve o pareamento de k motoristas com uma única busca no grafo reverso em lugar de k buscas independentes (redução de 10,7 vezes); e a validação por oráculo, na qual o Dijkstra — cuja corretude é demonstrada por indução — atua como gabarito numérico do A*.

SEÇÃO 3 | CONTEXTO E OBJETIVOS GERAIS

- (1) Ineficiência enfrentada.** Plataformas de mobilidade decidem a alocação de motoristas sob forte pressão de latência. A abordagem ingênua — ordenar candidatos por distância euclidiana e executar uma busca de rota completa para cada um — incorre em dois erros simultâneos. Primeiro, seleciona o motorista errado, porque a distância em linha reta ignora barreiras reais da malha: rios, vias expressas e sentido único. Em cenário controlado do projeto, um motorista situado a 275 m do passageiro leva 5,8 min para alcançá-lo, enquanto outro, a 548 m, leva 1,0 min. Segundo, desperdiça computação: k candidatos implicam k varreduras do grafo para atender a uma única solicitação.
- (2) Quem sofre o gargalo.** O passageiro, com tempo de espera maior que o necessário; o motorista parceiro, com deslocamento improdutivo não remunerado; e o operador da plataforma, com custo de infraestrutura proporcional ao desperdício e queda na taxa de conversão de corridas.
- (3) Objetivo principal.** Formalizar a malha viária como grafo ponderado direcionado e implementar uma camada algorítmica que devolva o motorista de menor tempo real de chegada e a rota ótima até o destino, com corretude demonstrada matematicamente e ganho de desempenho mensurado experimentalmente.

SEÇÃO 4 | ESCOPO INICIAL DO PROJETO

Objetivos Específicos (entregáveis técnicos)

- Modelagem matemática formal do domínio.** Malha viária como grafo ponderado direcionado $G = (V, E, w)$, com $w: E \rightarrow \mathbb{R}^+$ expressa em segundos de deslocamento, representada por lista de adjacência com custo de espaço $O(V + E)$; ciclo de vida da corrida modelado como grafo acíclico dirigido.
- Estruturação dos algoritmos e fluxos lógicos.** Rotina unificada de caminho mínimo, na qual o Dijkstra é o caso particular do A* com $h \equiv 0$; pareamento reverso many-to-one sobre o grafo invertido; quadtree para filtragem espacial; ordenação topológica de Kahn para o fluxo de estados.
- Implementação computacional em Python.** Pacote algoritmos/ com separação entre modelo matemático, rotinas de cálculo e cenários de avaliação.
- Construção de rotinas de validação e testes numéricos.** Suíte baseada em estratégia de oráculo, contemplando cenários de borda: peso negativo, alvo inalcançável, heurística inadmissível e sincronismo do grafo reverso.
- Geração de relatórios de saída e visualizações.** Rotina comparativa que mede nós expandidos e tempo de execução por algoritmo, acompanhada de documentação didática da fundamentação matemática.

CrITÉRIOS de Qualidade e Aceitação (parâmetros verificáveis)

Critério	Parâmetro mensurável	Situação
Exatidão matemática	Custo devolvido pelo A* idêntico ao do Dijkstra, com tolerância relativa de 1e-9, em 200 pares origem-destino sorteados.	Atendido
Cobertura de testes	Testes unitários cobrindo todas as funções de processamento matemático, incluindo casos de borda.	15/15
Documentação e complexidade	Código aderente à PEP 8, com complexidade assintótica declarada: $O((V + E) \log V)$ para a busca e $O(\log n)$ para a quadtree.	Em curso
Ganho de desempenho	Redução mínima de 3x nos nós expandidos na rota e de 5x no pareamento, preservando a solução ótima.	3,2x e 10,7x

SEÇÃO 5 | PILHA TECNOLÓGICA E ARQUITETURA

(1) Interpretador. Python 3.11.15.

(2) Bibliotecas. O núcleo matemático é implementado com a biblioteca padrão — `heapq` (fila de prioridade), `math` (fórmula de haversine) e `dataclasses` — uma vez que a disciplina exige implementação própria dos algoritmos. Em consequência, `NetworkX` é previsto exclusivamente como oráculo externo de validação, jamais como implementação. Complementam a pilha: `Matplotlib` (visualização de resultados), `NumPy` (consolidação de métricas), `FastAPI` (API REST), `MySQL` (persistência), `Leaflet.js` e `WebSocket` (camada de apresentação).

(3) Arquitetura modular. Quatro camadas, cada uma consumindo apenas a imediatamente inferior: apresentação, serviço/API, algoritmos e persistência. A camada de algoritmos é isolada e subdividida em modelo matemático (`grafo.py`), rotinas de cálculo (`busca.py` e `matching.py`), validação (`tests/`) e cenários de avaliação (`demo.py`).

SEÇÃO 6 | ORGANIZAÇÃO DO GRUPO E PAPÉIS TÉCNICOS

Integrante / RA	Papel no Grupo	Responsabilidade Principal na Modelagem
Kauan Sousa 8261113181	Líder de Projeto / Scrum Master	Coordenação geral, integração do repositório, cronograma e desenvolvimento da camada de serviço.
Bianca Murcela 8261104797	Modeladora Matemática	Formalização do grafo ponderado direcionado, prova de corretude por indução e análise de recorrência da quadtree.
Carolina Yukari 8261110898	Modeladora Matemática	Demonstração de admissibilidade e consistência da heurística; formalização da ordenação topológica do fluxo da corrida.
Carolina Chaves 826182449	Desenvolvedora Python	Codificação dos algoritmos principais: rotina unificada de caminho mínimo, grafo reverso e pareamento many-to-one.
Wildiner Lopes 8261102715	Engenheiro de Testes / QA	Baterias de validação numérica, estratégia de oráculo e cenários de borda; integração da camada de apresentação.
Gabryela Araujo 826199094	Documentadora Técnica	Documentação matemática, relatórios de desempenho, visualizações gráficas e interface do usuário.

SEÇÃO 7 | PLANEJAMENTO INICIAL E BACKLOG

Estão previstos quatro sprints de duas semanas até a 2ª parcial. O backlog inicial reúne as seguintes histórias técnicas: (a) modelar a malha viária e construir o grafo reverso com cache; (b) implementar a rotina unificada Dijkstra/A*; (c) implementar o pareamento reverso many-to-one; (d) implementar a quadtree de indexação espacial; (e) implementar a ordenação topológica de Kahn sobre o fluxo de estados da corrida; (f) expor a camada algorítmica por meio de API REST; e (g) integrar mapa e status em tempo real na camada de apresentação. A primeira sprint encontra-se concluída e entregou: o modelo de grafo com construção do reverso em cache e sincronismo de pesos, a rotina unificada de caminho mínimo, o pareamento reverso, a suíte de 15 testes numéricos e a rotina comparativa de desempenho.

SEÇÃO 8 | PRÓXIMOS PASSOS E CRONOGRAMA DA 2ª PARCIAL

Ação	Prazo
Implementar a quadtree de indexação espacial e integrá-la como filtro prévio do pareamento de motoristas.	05/10/2026
Implementar a ordenação topológica de Kahn e validar a aciclicidade do fluxo de estados da corrida.	19/10/2026
Expor a camada de algoritmos via API REST (FastAPI) com persistência em MySQL.	02/11/2026
Integrar a camada de apresentação (Leaflet.js e WebSocket) ao serviço de rotas.	16/11/2026
Consolidar o relatório de desempenho e a documentação matemática da 2ª parcial.	30/11/2026